

Control de Matriz LED HUB75 64x64 usando LiteX y RISC-V

Pablo Reinel

2025

Resumen

Este documento describe el desarrollo de un sistema completo basado en LiteX para controlar una matriz LED tipo HUB75 de 64x64 píxeles. Se implementó hardware en Verilog, incluyendo una FSM de control y un subsistema de memoria, y firmware en C para mostrar múltiples imágenes y secuencias animadas. El sistema soporta carga dinámica de firmware mediante Netboot por Ethernet, lo que permite modificar y probar el software sin reconfigurar la FPGA.

Índice

1. Introducción	3
2. Arquitectura del Sistema	3
3. Descripción del Módulo Verilog	4
3.1. Estructura del controlador	4
3.2. Formato de almacenamiento de píxeles	4
4. Firmware en C	4
4.1. Funciones principales	4
4.1.1. Delay por hardware	4
4.1.2. Carga de imagen	5
4.1.3. Borrado de pantalla	5
4.2. Secuencia principal	5
5. Conversión de Imágenes	6
6. Netboot por Ethernet	6
7. Preguntas Potenciales del Profesor	6
7.1. ¿Por qué la imagen se divide en dos mitades?	6
7.2. ¿Por qué se invierten los colores?	6
7.3. ¿Qué garantiza que no haya colisiones de lectura/escritura?	6

7.4. ¿Cómo aumentar la velocidad del refresco?	6
7.5. ¿Por qué 2048 palabras?	7
8. Conclusiones	7

1. Introducción

El objetivo del proyecto es implementar un controlador funcional para una matriz LED HUB75 de 64x64 píxeles, utilizando un SoC RISC-V generado por LiteX. Para ello se integraron componentes en Verilog y software en lenguaje C, permitiendo cargar, actualizar y animar imágenes directamente desde la memoria de sistema.

El flujo de desarrollo emplea Netboot por Ethernet, disminuyendo el tiempo de iteración al evitar la reprogramación del bitstream FPGA en cada cambio.

2. Arquitectura del Sistema

La arquitectura general del sistema consta de:

- CPU RISC-V generada por LiteX.
- Periférico mapeado en memoria `led_panel0` conectado al módulo Verilog.
- Memoria interna para almacenar 2048 palabras de 24 bits.
- FSM de control encargada del multiplexado y la temporización del panel.
- Firmware en C ejecutado desde SDRAM.

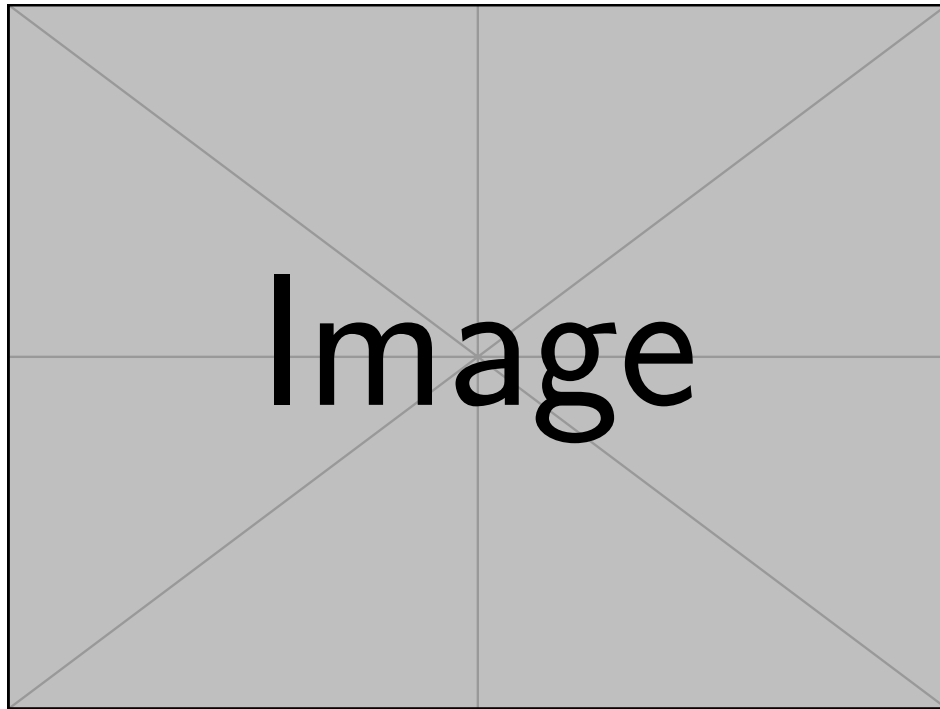


Figura 1: Diagrama conceptual del sistema basado en LiteX y el panel HUB75.

3. Descripción del Módulo Verilog

3.1. Estructura del controlador

El módulo `led_panel_4k.v` integra:

- Contador de columnas (COL).
- Contador de filas multiplexadas (ROW).
- FSM encargada del ciclo HUB75 (Latch, OE, Clock).
- Memoria interna definida en `memory.v`.
- Multiplexor RGB que combina dos píxeles por palabra.

3.2. Formato de almacenamiento de píxeles

Cada palabra de memoria almacena dos píxeles (fila superior e inferior). El formato usado por el panel es:

BYTE1: $R_1[3 : 0] \ G_1[3 : 0]$

BYTE2: $B_1[3 : 0] \ R_2[3 : 0]$

BYTE3: $G_2[3 : 0] \ B_2[3 : 0]$

Este empaquetado permite sincronizar la actualización de filas superiores e inferiores simultáneamente, característica de los paneles HUB75.

4. Firmware en C

4.1. Funciones principales

4.1.1. Delay por hardware

```
void my_busy_wait(unsigned int ms) {
    timer0_en_write(0);
    timer0_reload_write(0);
    timer0_load_write(CONFIG_CLOCK_FREQUENCY/1000*ms);
    timer0_en_write(1);
    timer0_update_value_write(1);
    while(timer0_value_read()) timer0_update_value_write(1);
}
```

4.1.2. Carga de imagen

```
void modificar_imagen(const uint32_t *in){
    for (int i = 0; i < 2048; i++){
        led_panel0_mem_w_address_write(i);
        led_panel0_we_a_write(1);
        led_panel0_mem_w_data_write(in[i]);
        led_panel0_we_a_write(0);
        my_busy_wait(1);
    }
}
```

4.1.3. Borrado de pantalla

```
void limpiar_imagen(){
    for (int i = 0; i < 2048; i++){
        led_panel0_mem_w_address_write(i);
        led_panel0_we_a_write(1);
        led_panel0_mem_w_data_write(0);
        led_panel0_we_a_write(0);
        my_busy_wait(1);
    }
}
```

4.2. Secuencia principal

```
while(1){
    modificar_imagen(linux);
    my_busy_wait(1000);
    limpiar_imagen();
    my_busy_wait(1000);

    modificar_imagen(lufy);
    my_busy_wait(1000);
    limpiar_imagen();
    my_busy_wait(1000);

    modificar_imagen(principito);
    my_busy_wait(1000);
    limpiar_imagen();
    my_busy_wait(1000);

    ...
}
```

Este ciclo actúa como un **slideshow animado**.

5. Conversión de Imágenes

Las imágenes se convierten mediante un script Python que:

- Redimensiona a 64x64.
- Invierte canales RGB debido al orden físico BGR del panel.
- Reduce precisión a 4 bits por canal.
- Genera 2048 palabras de 24 bits empaquetados.

6. Netboot por Ethernet

El firmware se carga con:

```
make firmware  
lites_term --netboot firmware.bin
```

Ventajas:

- Cambios instantáneos sin reprogramar FPGA.
- Flujo de trabajo más rápido.
- Permite pruebas repetidas de animación.

7. Preguntas Potenciales del Profesor

7.1. ¿Por qué la imagen se divide en dos mitades?

El panel refresca dos filas simultáneamente, por lo que cada palabra describe un píxel superior y uno inferior.

7.2. ¿Por qué se invierten los colores?

El panel trabaja en orden BGR y el Verilog lo interpreta así.

7.3. ¿Qué garantiza que no haya colisiones de lectura/escritura?

La FSM lee en `posedge clk` y la CPU escribe en `negedge clk`.

7.4. ¿Cómo aumentar la velocidad del refresco?

Con doble buffer en memoria interna.

7.5. ¿Por qué 2048 palabras?

$$64 \times 32 = 2048$$

Ya que cada palabra contiene dos filas.

8. Conclusiones

El proyecto permitió integrar hardware digital, software embebido y comunicación mediante Ethernet. El sistema final es capaz de mostrar múltiples imágenes y animaciones en un panel HUB75 controlado por una plataforma LiteX/RISC-V. Además, el uso de Netboot permitió un flujo de trabajo eficiente y altamente flexible.